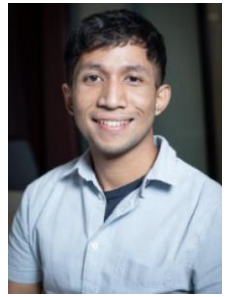
Bioinformatics Shared Resource
Cold Spring Harbor Laboratory

Library and Archives on Bioinformatics Training Hours

Hands-on Machine Learning for Beginners

Jackie Gunther
Research Engagement and Data Curator
Library and Archives

June 15th and 16th, 2026
Hawkins Conference Room



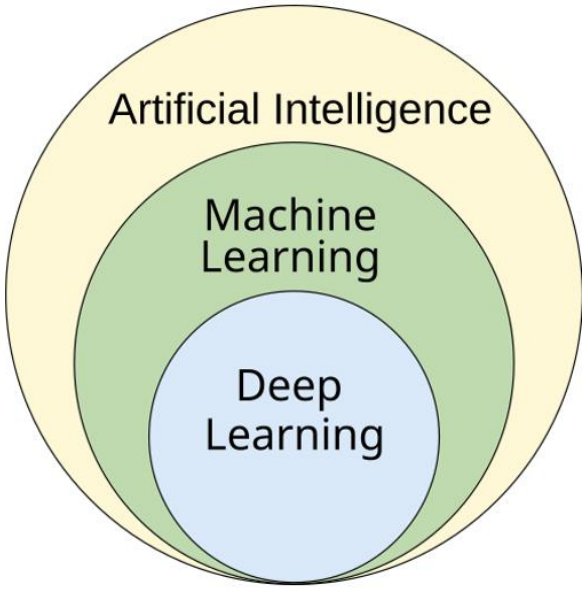
Rad Utama
Director

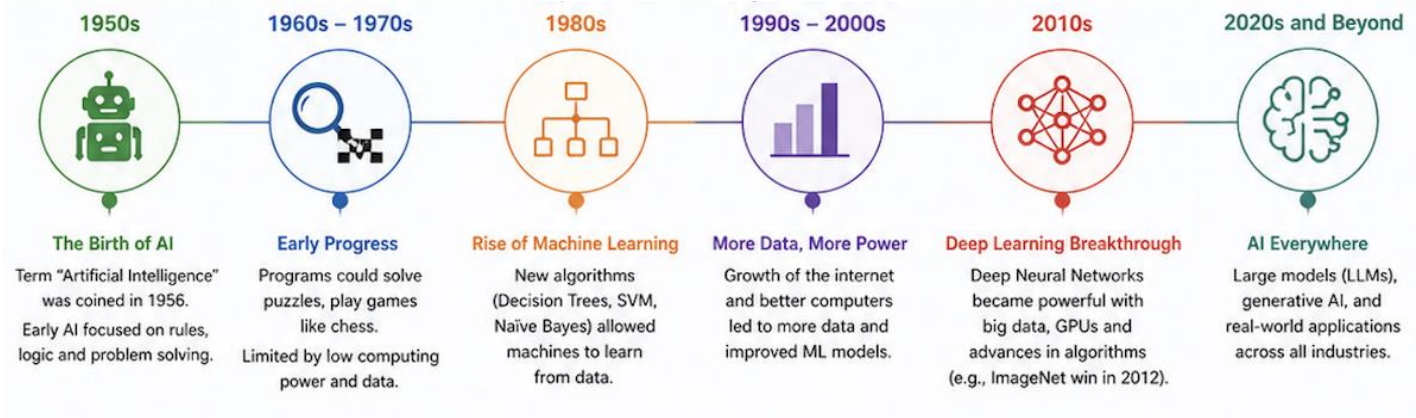
James Rouse
Analyst

Agenda

1. *Introduction to AI, Machine Learning, and Deep Learning*
2. *Data Preprocessing and Best Practices*
3. *Dimensionality Reduction: PCA and UMAP/tSNE*
4. *Regression: Linear and Logistic*
5. *k-Clustering: k-NN and k-means*
6. *Hierarchical Clustering and SVM*
7. *Tree Classifier: Decision Tree and Random Forest*
8. *Neural Networks*

Introduction





Level	What it means
AI	Machines that can perform tasks that normally require human intelligence. Includes rules-based systems and learning-based systems.
ML	Machines learn from data without being explicitly programmed. Focuses on finding patterns and making predictions.
DL	Uses deep (multi-layer) neural networks to learn complex patterns from large amounts of data. Excels in unstructured data like images, text, audio.

Application

Input (X)	Output (Y)	Application
email	spam? (0/1)	spam filtering
audio	text transcripts	speech recognition
English	Spanish	machine translation
ad, user info	click? (0/1)	online advertising
image, radar info	position of other cars	self-driving car

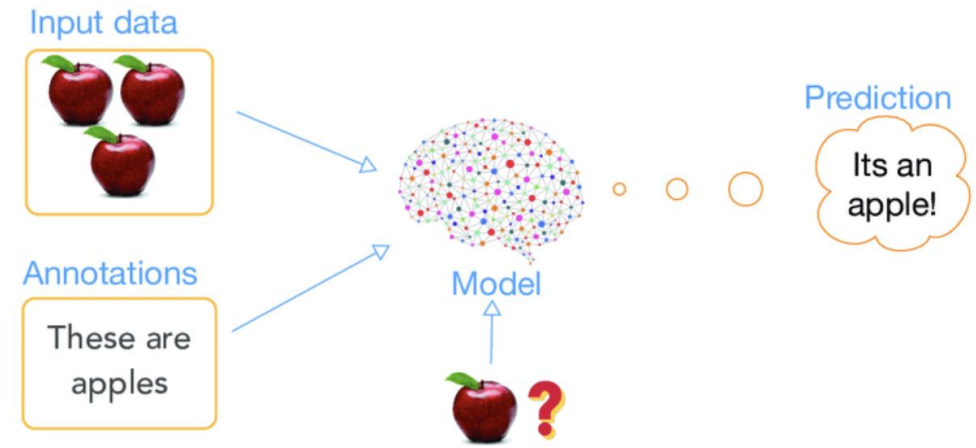
Machine Learning

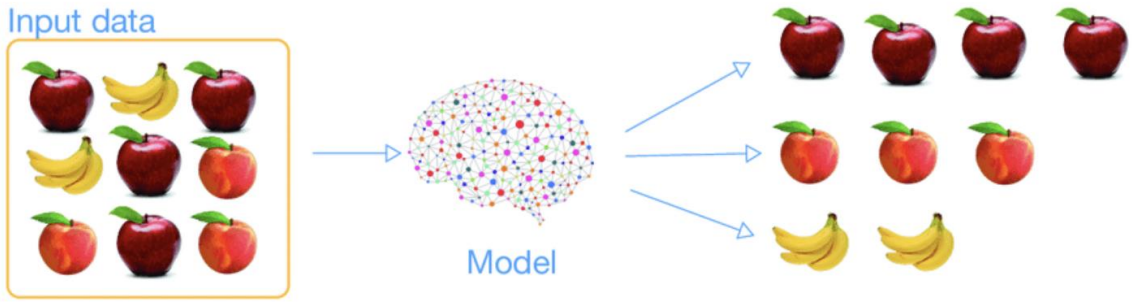
What is Machine Learning ?

"the field of study that gives computers the ability to learn without being explicitly programmed."
Arthur Samuel, 1959

Types of Machine Learning

- Supervised Learning (Prediction)
  - Regression (Predicts Number)
  - Classification (Predicts Category)
- Unsupervised Learning (Pattern)
  - Clustering
  - Dimensionality Reduction
  - Outlier/Anomaly Identification





Methods: Supervised (Labeled Data) vs Unsupervised (Unlabeled data)

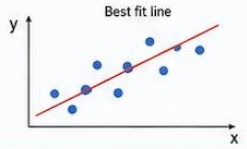
SUPERVISED LEARNING

Learn from labeled data
(input X → known output y)

1 LINEAR REGRESSION

Predicts a continuous numeric value.

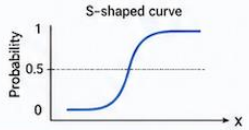
Example: Predict house price



2 LOGISTIC REGRESSION

Predicts a probability or class (0 or 1).

Example: Spam (yes/no)



3 k-NEAREST NEIGHBORS (kNN)

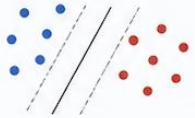
Predicts based on the labels of the k most similar points.

Example: Classify a new customer

4 SUPPORT VECTOR MACHINE (SVM)

Finds the best boundary (hyperplane) that separates classes with the maximum margin.

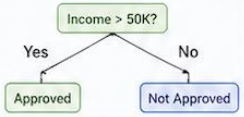
Example: Classify images



5 DECISION TREE

Splits data into branches based on feature rules.

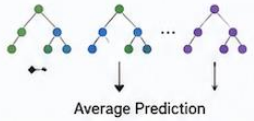
Example: Predict loan approval



6 RANDOM FOREST

Uses many decision trees and averages their predictions for better accuracy.

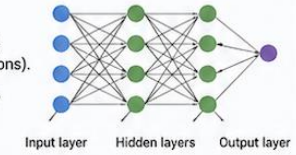
Example: Predict customer churn



7 NEURAL NETWORKS

Models complex patterns using layers of connected nodes (neurons).

Example: Image recognition, NLP



VS

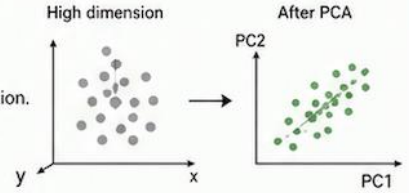
UNSUPERVISED LEARNING

Find patterns or structure
in unlabeled data (input X only)

1 PCA (PRINCIPAL COMPONENT ANALYSIS)

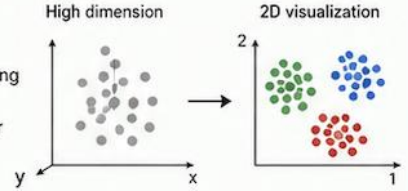
Reduces dimensions while keeping important information.

Useful for visualization and noise reduction.



2 UMAP / t-SNE

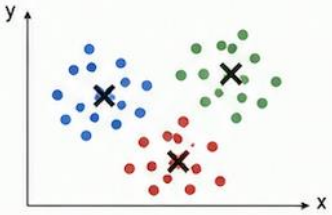
Reduces dimensions for visualization while preserving local structure (UMAP is faster; t-SNE very good for visual clusters).



3 K-MEANS CLUSTERING

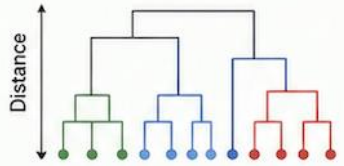
Groups data into K clusters based on similarity.

Example: Customer segmentation



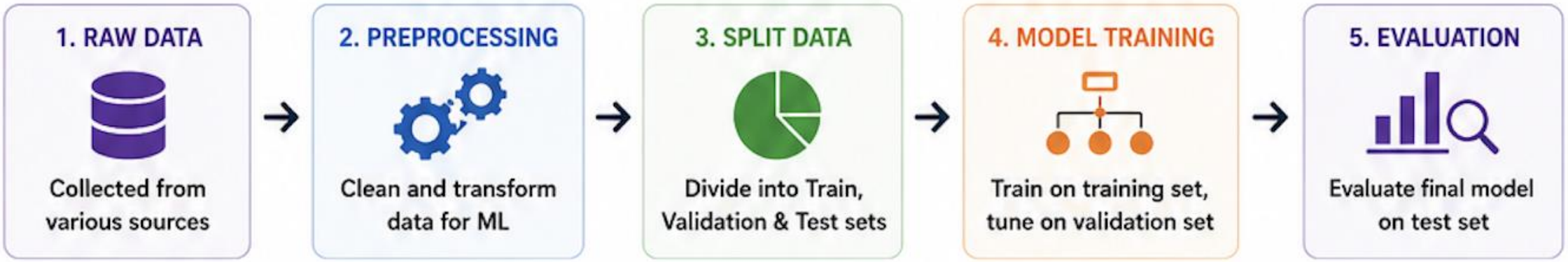
4 HIERARCHICAL CLUSTERING

Builds a tree (dendrogram) of clusters without needing to choose K in advance.



Training Framework

Framework


1. RAW DATA
Collected from various sources
2. PREPROCESSING
Clean and transform data for ML
3. SPLIT DATA
Divide into Train, Validation & Test sets
4. MODEL TRAINING
Train on training set, tune on validation set
5. EVALUATION
Evaluate final model on test set

Data Preprocessing

1. PREPROCESSING STEPS

1.1 UNDERSTAND THE DATA

- Check data types
- Summary statistics
- Visualize distributions
- Detect problems early

1.2 HANDLE MISSING VALUES

- Remove rows/columns (with many missing)
- Impute missing values (mean, median, mode, or advanced methods)

1.3 HANDLE DUPLICATES

- Find and remove duplicate rows
- Keep the most relevant records

1.4 HANDLE OUTLIERS

- Detect using plots or statistics (IQR, Z-score)
- Cap, transform or remove if needed

1.5 ENCODE CATEGORICAL VARIABLES

Color	Red	Blue	Green
Red	1	0	0
Blue	0	1	0
Green	0	0	1

- Label Encoding (ordinal)
- One-Hot Encoding (nominal)

1.6 FEATURE SCALING

- Standardization (Z-score)
- Normalization (Min-Max)
- Needed for algorithms like kNN, SVM, Neural Networks

1.7 FEATURE ENGINEERING (OPTIONAL)

- Create informative features
- Combine or transform existing features

Goal: Clean, consistent, and informative data that helps the model learn better.

3. PREPROCESSING BEST PRACTICES (IMPORTANT!)

Fit prepocessing steps using ONLY the train set → Apply the same transformations to validation and test sets → This prevents data leakage and ensures realistic evaluation → Always build a reproducible preprocessing pipeline

DATA LEAKAGE EXAMPLE (AVOID!)

✗ Don't compute mean, median, or scaling using the whole dataset before splitting.

✓ Compute using TRAIN SET only, then apply to others.

Data Splitting

2. SPLIT THE DATA

TRAIN SET (≈ 70%)

- Used to train the model.
- Model learns patterns from this data.

VALIDATION SET (≈ 15%)

- Used to tune hyperparameters and make decisions during model development.
- (Not used for final training)

TEST SET (≈ 15%)

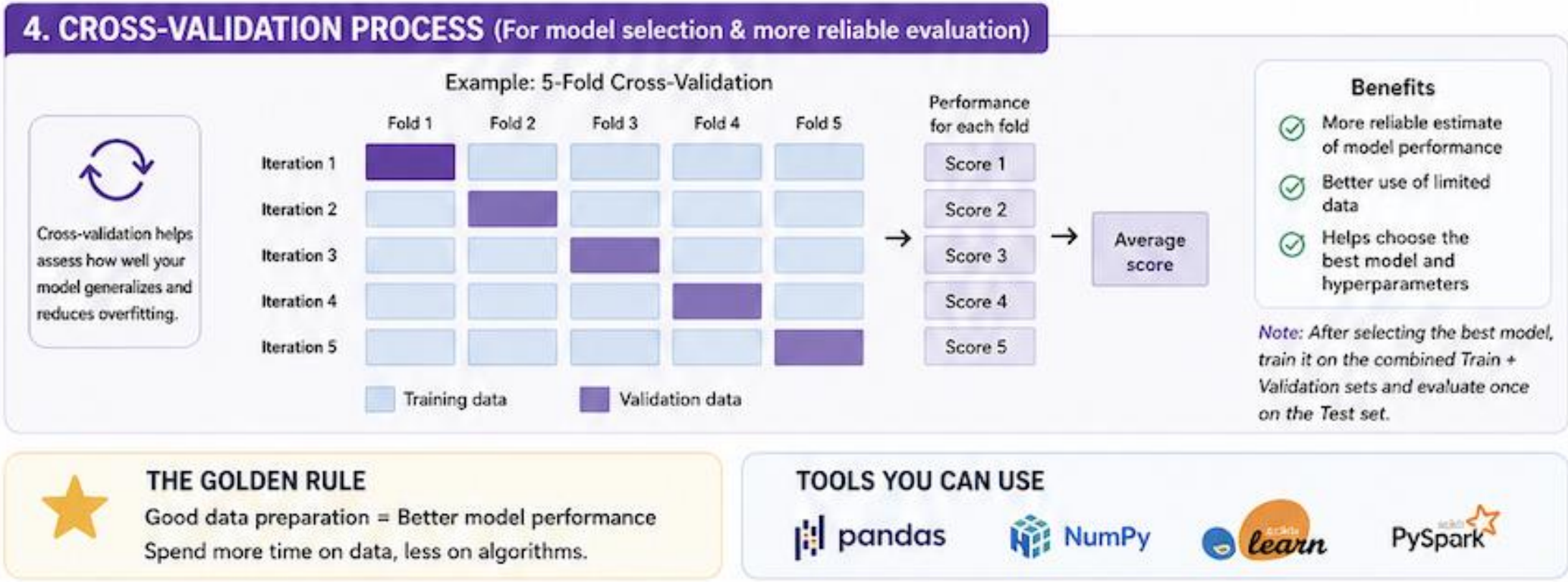
Used only once at the end to evaluate the final model performance on unseen data.

Common Ratios: 70/15/15 or 80/10/10 (adjust based on dataset size)

TIPS

- Use stratified split for classification (maintains class distribution).
- For time series data, use chronological split.
- Keep test set completely untouched until the end.

Cross Validation

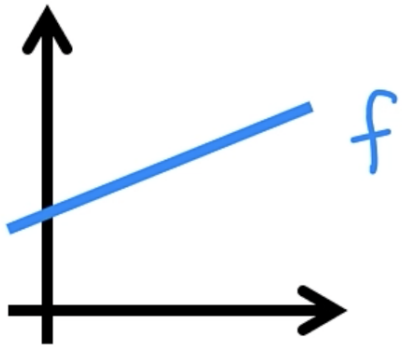


Model Training

Model Training

- model:

$$f_{w,b}(x) = wx + b$$



- parameters:

$$w, b$$

- cost function:

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$$

- goal:

$$\underset{w,b}{\text{minimize}}\, J(w,b)$$

Cost Function

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

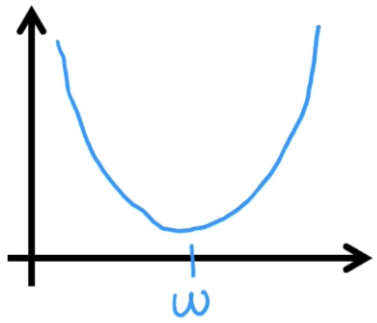
$\hat{y}^{(i)}$: error

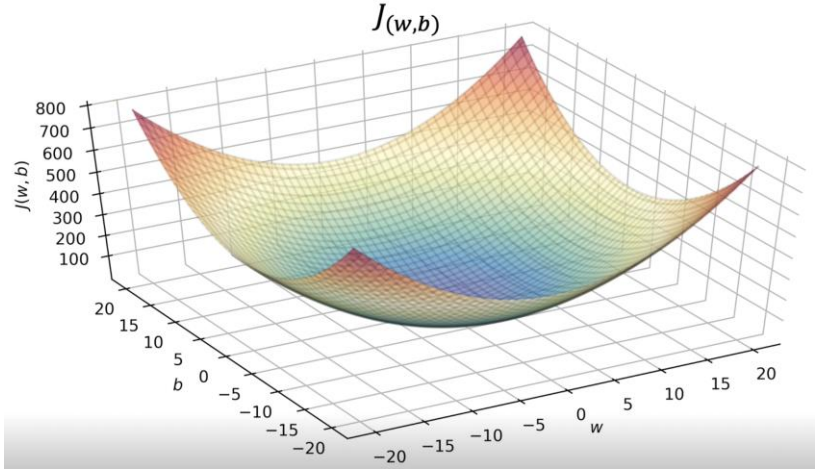
m = number of training examples

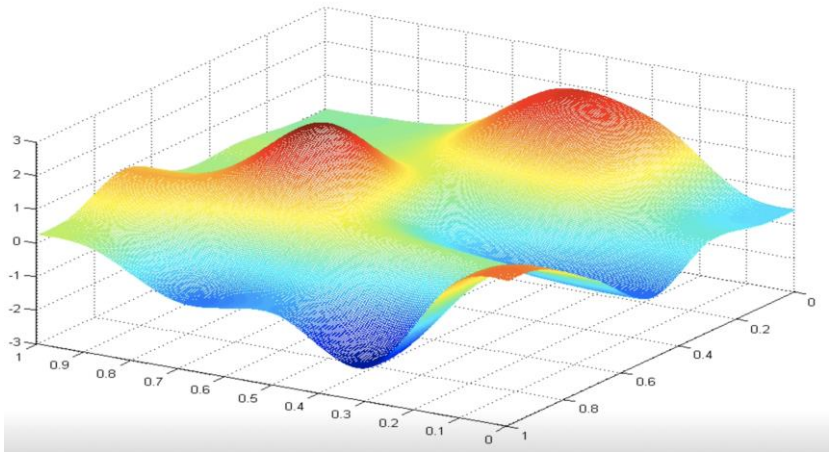
$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}\left(x^{(i)}\right) - y^{(i)}\right)^2$$

J

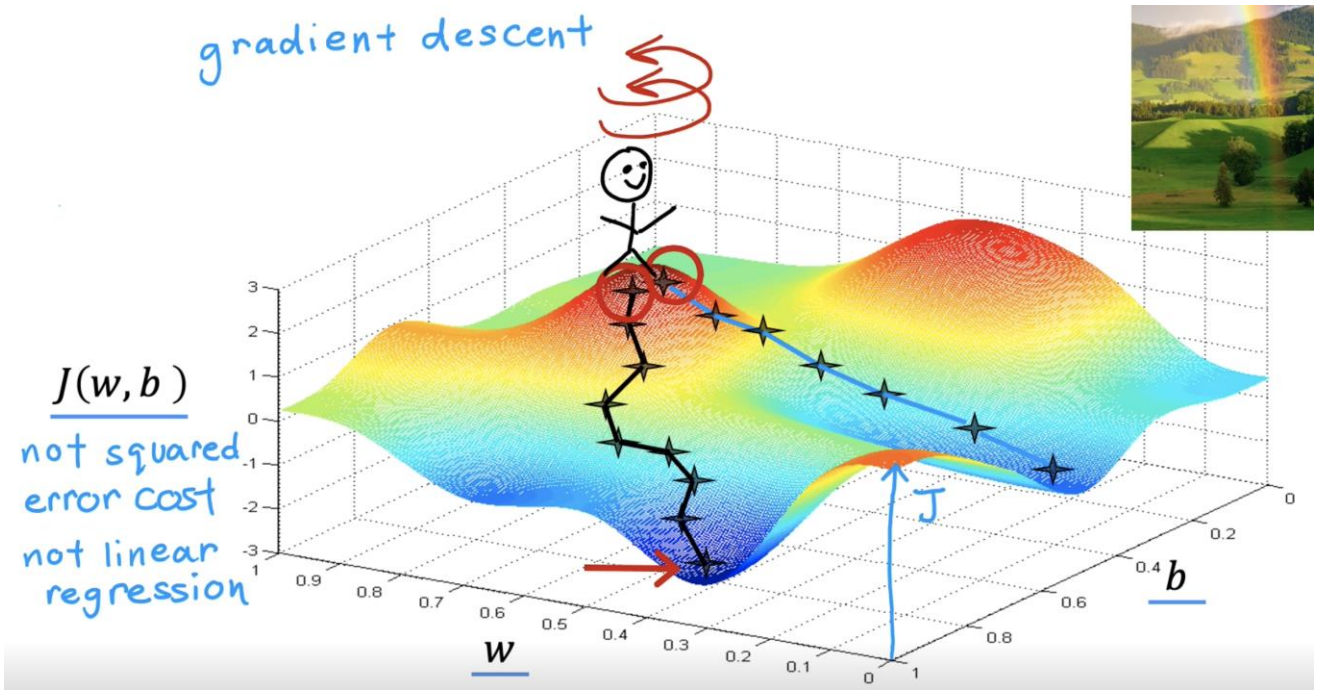
(function of w, b)







Minimizing Cost with Gradient Descent



Learning Rate
(How Big is Your Step)

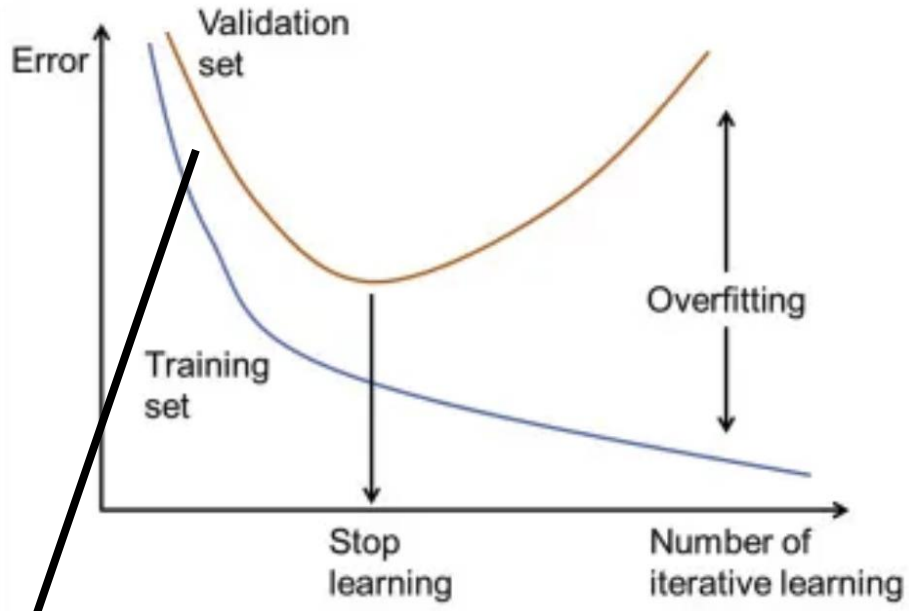
Correct: Simultaneous update

$$tmp\_w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

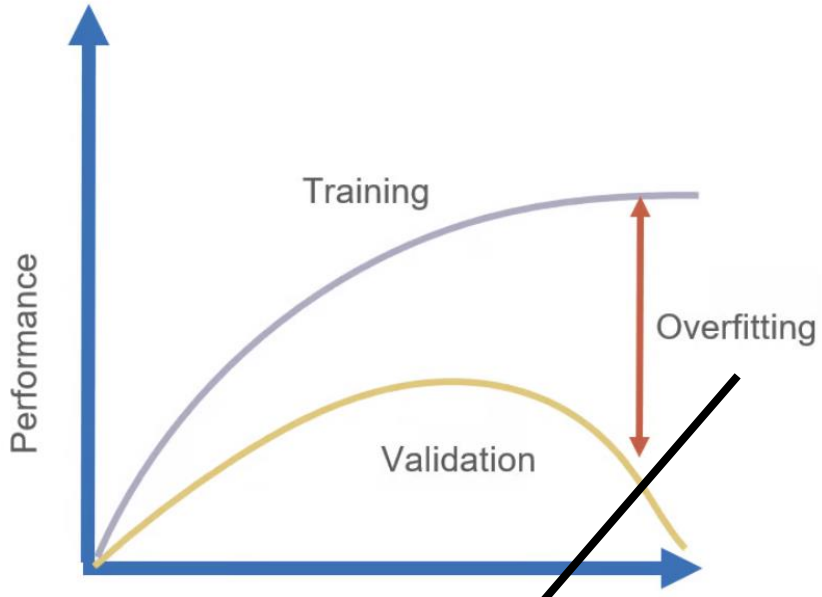
$$tmp\_b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

Curse of Overfitting & Model Overcomplexity

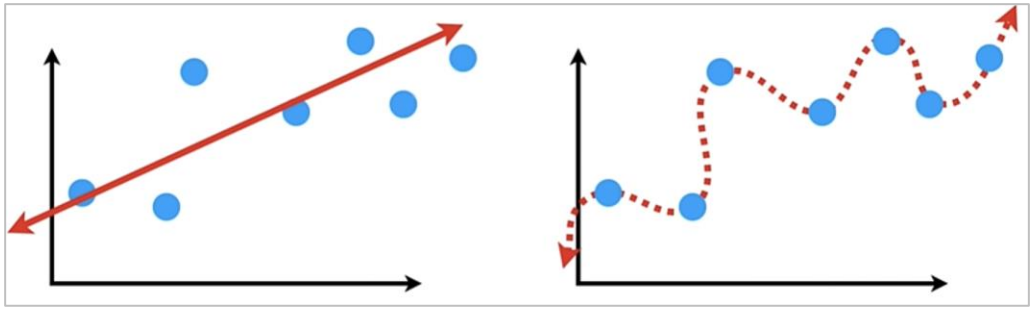
Cost



Accuracy



Underfitting



Performance Evaluation

Evaluation

1. CLASSIFICATION METRICS (for yes/no or multi-class problems)

CONFUSION MATRIX

Table showing actual vs predicted

Actual \ Predicted	Positive (1)	Negative (0)
Positive (1)	TP — True Positive (correct positive)	FN — False Negative (missed positive)
Negative (0)	FP — False Positive (wrong positive)	TN — True Negative (correct negative)

TP: True Positive (actual 1, predicted 1)

TN: True Negative (actual 0, predicted 0)

FP: False Positive (actual 0, predicted 1)

FN: False Negative (actual 1, predicted 0)

ACCURACY

How often am I right?

$$\frac{(TP + TN)}{TP + TN + FP + FN}$$

✓ Good for balanced classes

✗ Not good for imbalanced classes

PRECISION

When I predict positive, how often am I correct?

$$\frac{TP}{TP + FP}$$

✓ High precision = few false positives

RECALL (SENSITIVITY)

How many real positives did I catch?

$$\frac{TP}{TP + FN}$$

✓ High recall = few false negatives

F1 SCORE

Balance between precision & recall

$$\frac{2 \times Precision \times Recall}{Precision + Recall}$$

✓ Good for imbalanced data

SPECIFICITY (TRUE NEGATIVE RATE)

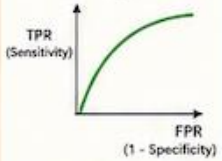
How well do I detect negatives?

$$\frac{TN}{TN + FP}$$

Complement of recall. High = few false positives

ROC-AUC

Can the model rank positives higher than negatives?

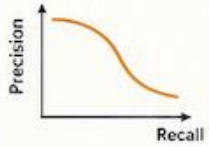


Range: 0.5 (random) to 1 (perfect)

Good for ranking ability, any threshold

PR-AUC

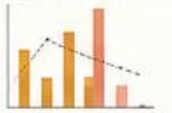
How good is the precision–recall trade-off?



More informative than ROC-AUC when positives are rare (imbalanced data)

LOG LOSS (CROSS-ENTROPY)

How confident and correct are the probabilities?



Penalizes confident but wrong predictions heavily

QUICK GUIDE

- Rare events (imbalanced data) → Use Recall, Precision, F1, PR-AUC
- False positives expensive → Focus on Precision
- False negatives expensive → Focus on Recall
- General purpose (balanced data) → Accuracy, ROC-AUC
- Probabilistic models → Use Log Loss to evaluate probability quality

Evaluation

2. REGRESSION METRICS (for predicting numbers)

MAE
(Mean Absolute Error)

Average absolute mistake

$$\frac{1}{n}\sum_{i=1}^{n}|y_i' - \hat{y}_i|$$

- ✓ Easy to interpret
- ✓ Robust to outliers

MSE
(Mean Squared Error)

Average squared mistake

$$\frac{1}{n}\sum_{i=1}^{n}(y_i' - \hat{y}_i)^2$$

- ✓ Penalizes large errors more
- ✗ Sensitive to outliers

RMSE
(Root Mean Squared Error)

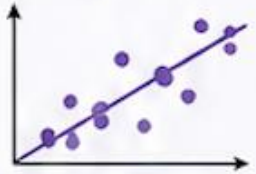
Square root of MSE (error in original units)

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i' - \hat{y}_i)^2}$$

- ✓ Most commonly reported
- ✓ Interpretable in original units

R^2 (R-SQUARED)
(Coefficient of Determination)

How much variance did I explain?

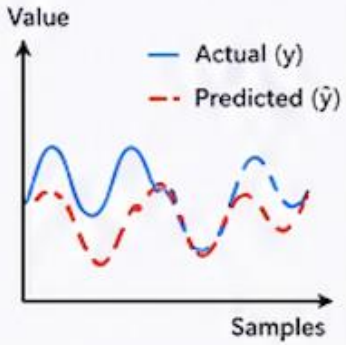


- ✓ Good for overall model fit

y_i = actual value

$\hat{y}_i$ = predicted value

n = number of samples



3. HOW TO CHOOSE THE RIGHT METRIC

CLASSIFICATION

Balanced Classes	Imbalanced Classes
Use Accuracy + ROC-AUC	Use F1 Score + PR-AUC

COST-SENSITIVE PROBLEMS

False Positives are expensive	False Negatives are expensive
Focus on Precision	Focus on Recall

REGRESSION

Need simple, interpretable errors	Want to penalize big errors more
Use MAE	Use RMSE

GENERAL TIP

Always understand your problem, the data, and the cost of different types of errors.

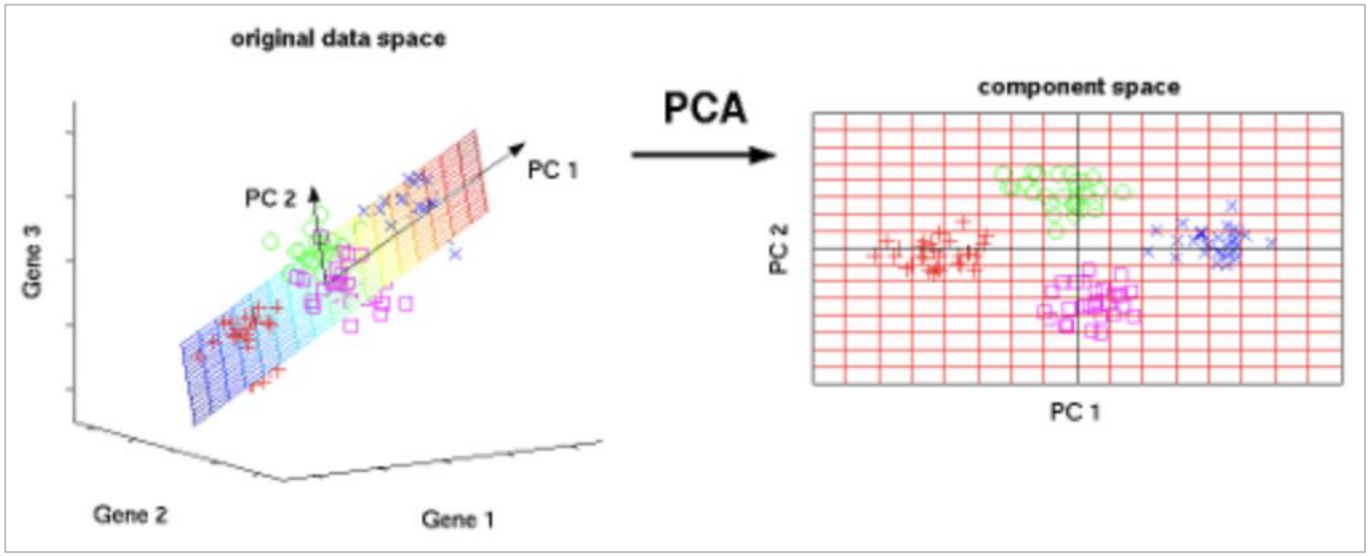
No single metric tells the whole story!

★ **PRO TIP:** Look at multiple metrics, not just one. Also check the confusion matrix / error analysis to truly understand your model.

Popular Methods, Part 1
Machine Learning

Principal Component Analysis (PCA)

1. Unsupervised
2. Dimensionality Reduction
3. Linear Projection/Combination
4. Axis Rotation
5. Maximize Variance



Covariance Matrix

$$\Sigma = \frac{1}{n-1} Z^T Z$$

Eigenvalues

$$\det(\Sigma - \lambda I) = 0$$

Variance of each PC

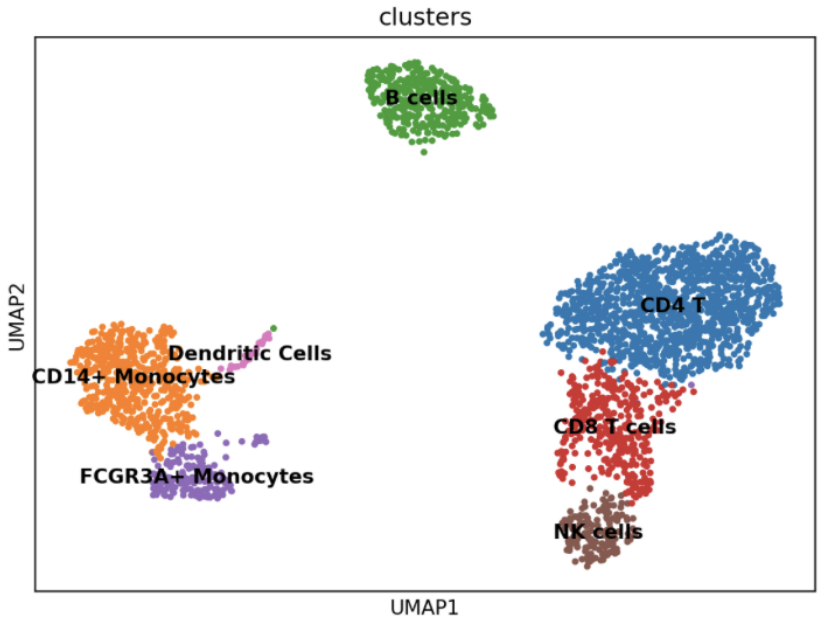
Eigenvectors

$$(\Sigma - \lambda I)v = 0$$

Composition of each PC

Uniform Manifold Approx. & Projection (UMAP)

1. Unsupervised
2. Dimensionality Reduction
3. Non-Linear Projection
4. Preserves Local Structure
5. Stochastic



Original
High Dimension Similarity Prob. (Fixed)

$$p_{j|i} = \exp\left(-\frac{\max(0, d(X_i, X_j) - \rho_i)}{\sigma_i} \right)$$

Projected
Low Dimension Similarity Prob. (Dynamic)

$$q_{ij} = (1 + a \cdot ||y_i - y_j||^{2b})^{-1}$$

Minimum Distance (min_dist)

Cost Function (Cross Entropy)

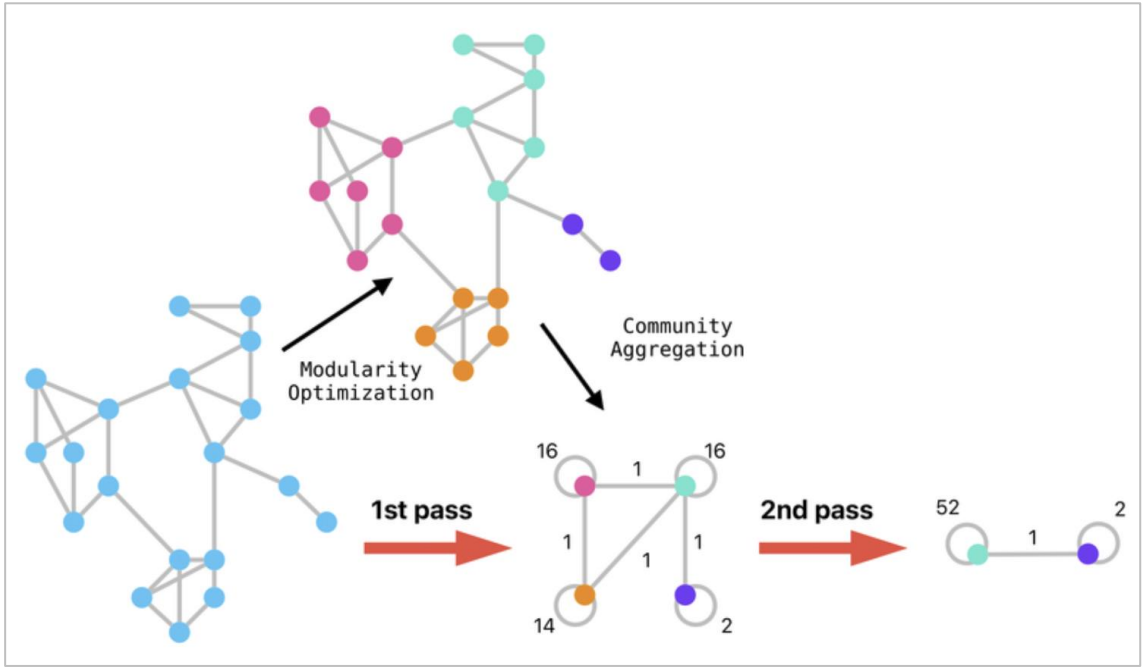
$$C = \sum_{i \neq j} \left(p_{ij} \log \frac{p_{ij}}{q_{ij}} + (1 - p_{ij}) \log \frac{1 - p_{ij}}{1 - q_{ij}} \right)$$

Pulling Force

Pushing Force

Louvain and Leiden Clustering (Graph-Based)

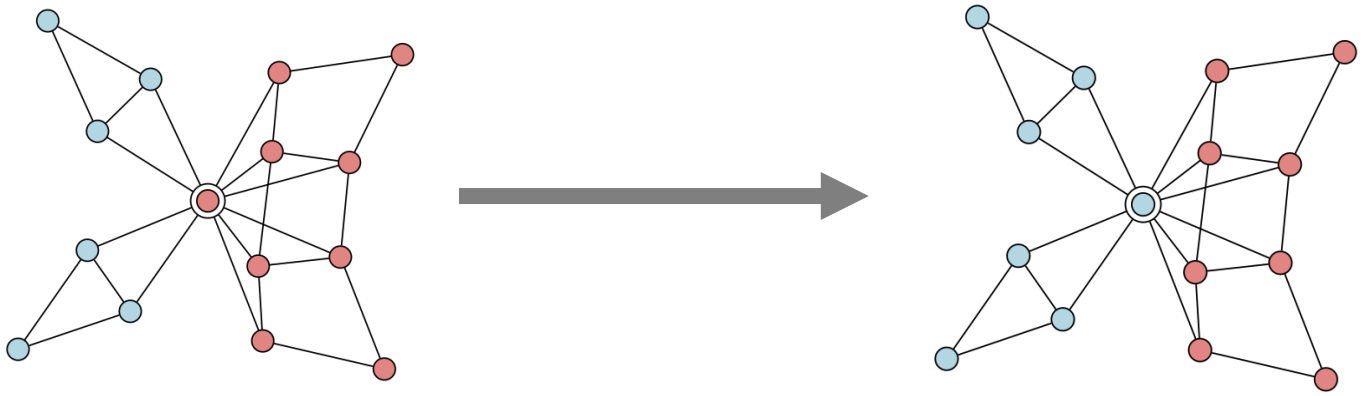
1. Unsupervised
2. Clustering
3. Graph-Based
4. Maximize Modularity



Modularity

$$Q = \frac{1}{2m}\sum_{i,j}\left(A_{ij} - \frac{k_i k_j}{2m} \right)\delta(c_i, c_j)$$

Leiden Improvement -> Prevent Breaking/Disconnecting Communities

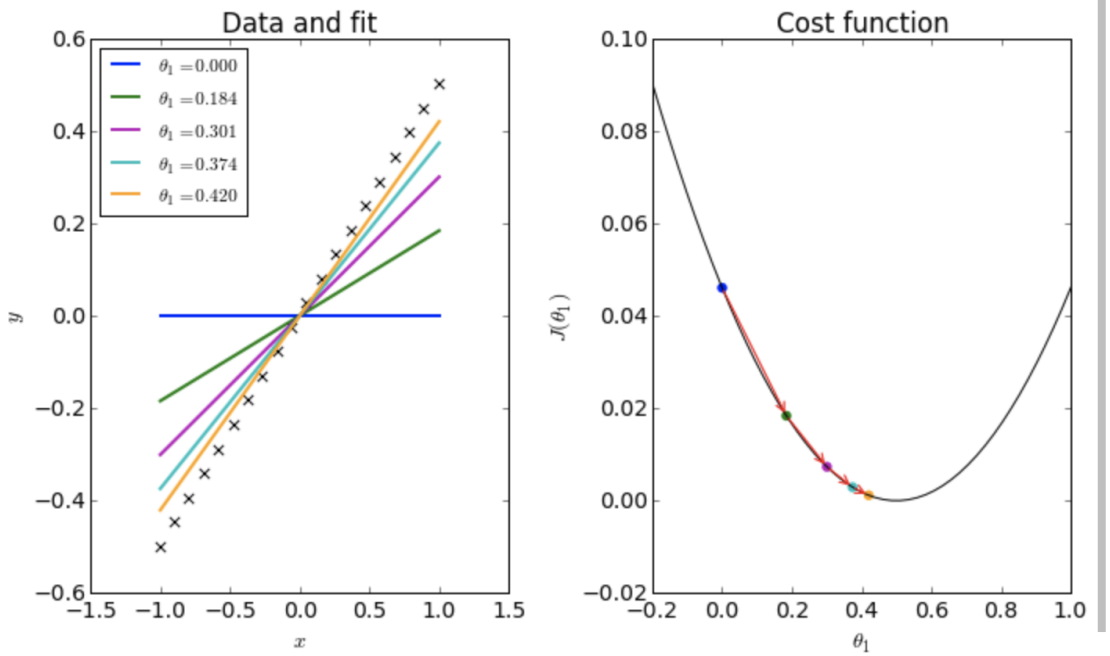


Linear and Logistic Regressions

1. Supervised
2. Regressions (numbers)
3. Linear Function
4. Correlation and R-squared
5. Multiple vs Multivariate

Linear

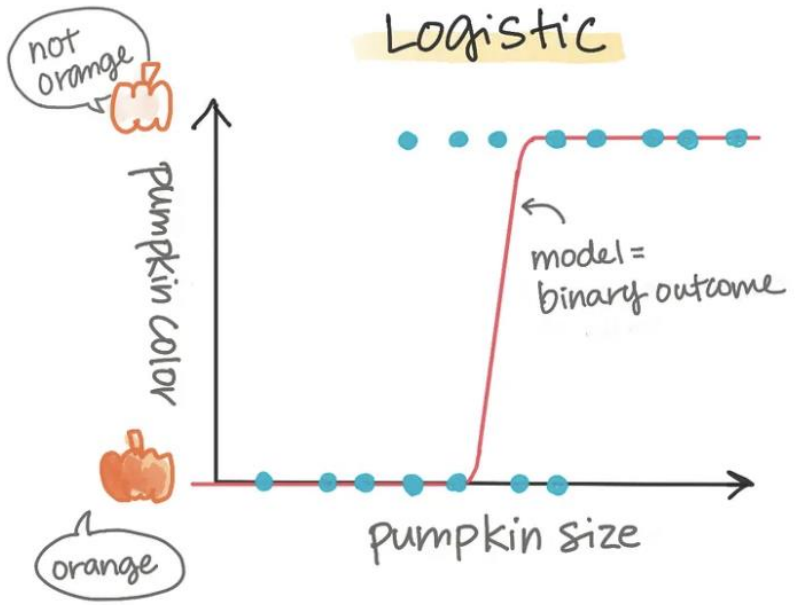
$$f_{w,b}(x) = wx + b$$



Logistic

1. Supervised
2. Classifications (Categories)
3. Logistic Function
4. Odds Ratio
5. Multiple vs Multivariate

$$p = \frac{1}{1 + e^{-(b_0 + b_1 x)}}$$



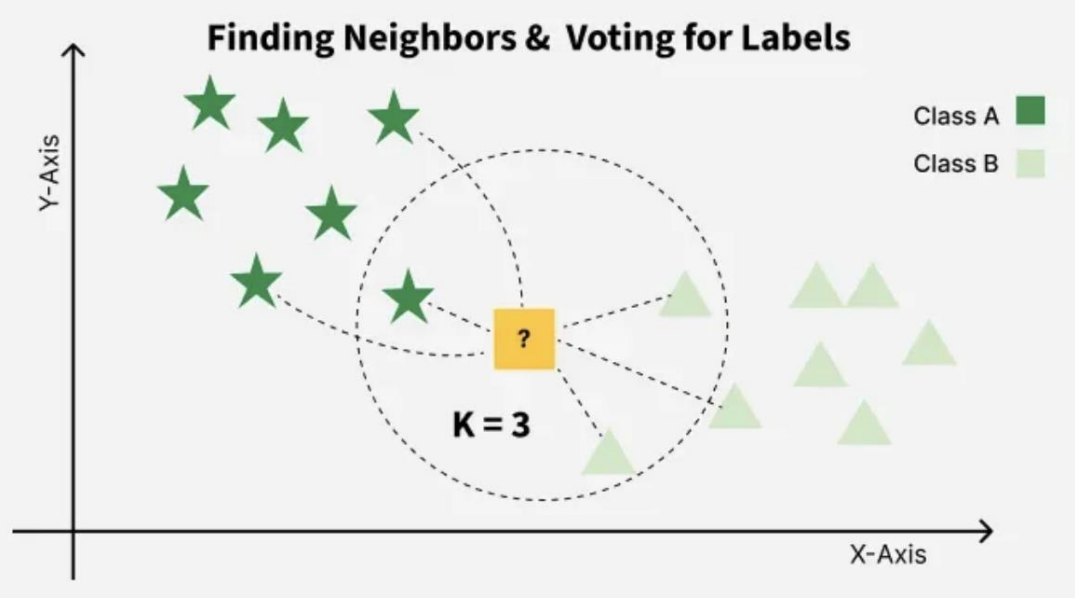
Popular Methods, Part 2
Machine Learning

k-NN (Nearest neighbor) and k-means Clustering

1. Supervised
2. Classifications
3. Calculate Distance
4. Identify k Nearest Neighbors
5. Voting for Final Label

k-NN

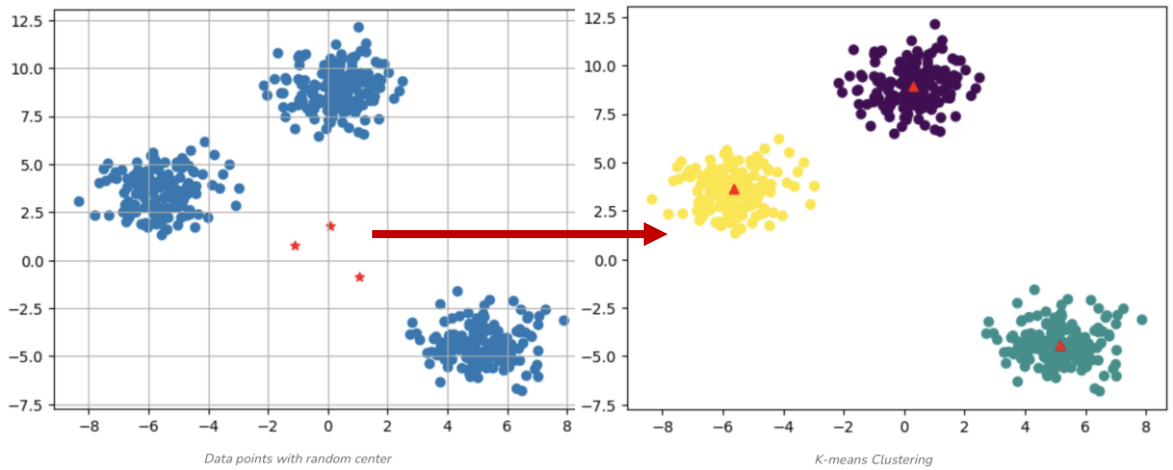
Distance: $d(x, X_i) = \sqrt{\sum_{j=1}^{n} (x_j - X_{ij})^2}$



k-means

1. Unsupervised
2. Clustering
3. Start Random k Centroids
4. Assign Points to Nearest Centroid
5. Recalculate Centroids

Minimize Inertia: $J = \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$

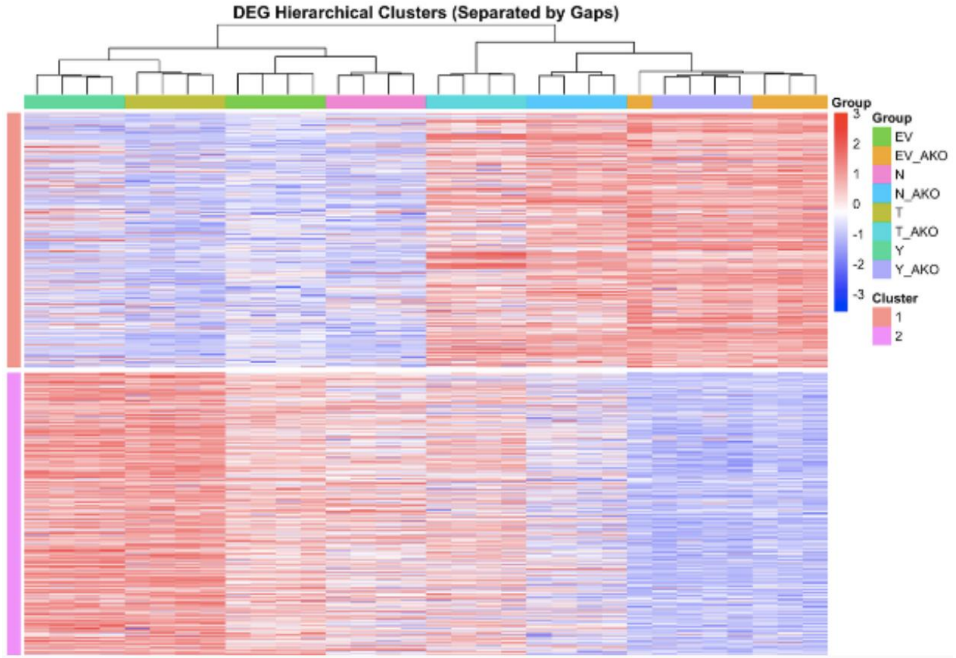


Hierarchical Clustering and Support Vector Machine (SVM)

1. Unsupervised
2. Clustering
3. Calculate Distance
4. Cluster Linkage
5. Dendrogram Tree

Hierarchical

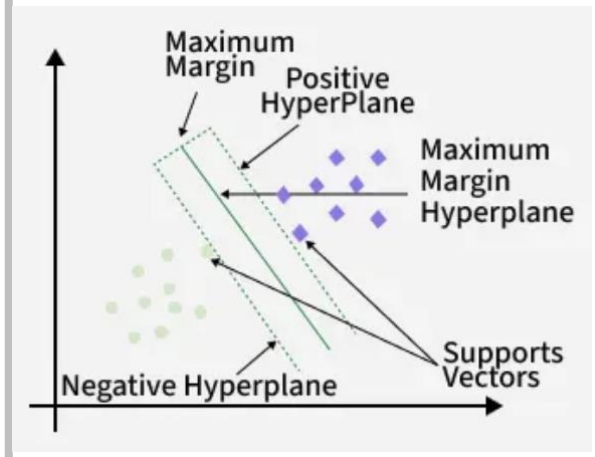
Distance: $d(x, X_i) = \sqrt{\sum_{j=1}^{n} (x_j - X_{ij})^2}$

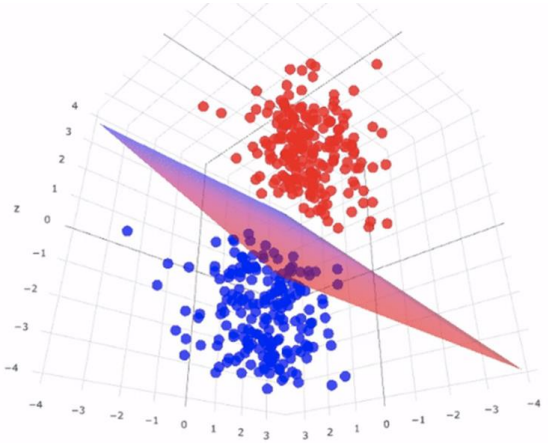


SVM

1. Supervised
2. Classification
3. Best Way to Cut/Max Margin
4. Hyperplane (Dimension - 1)
5. Define Points as Support Vectors

$$\text{Objective Function} = \left(\frac{1}{\text{margin}}\right) + \lambda \sum \text{penalty}$$





Decision Tree and Random Forest

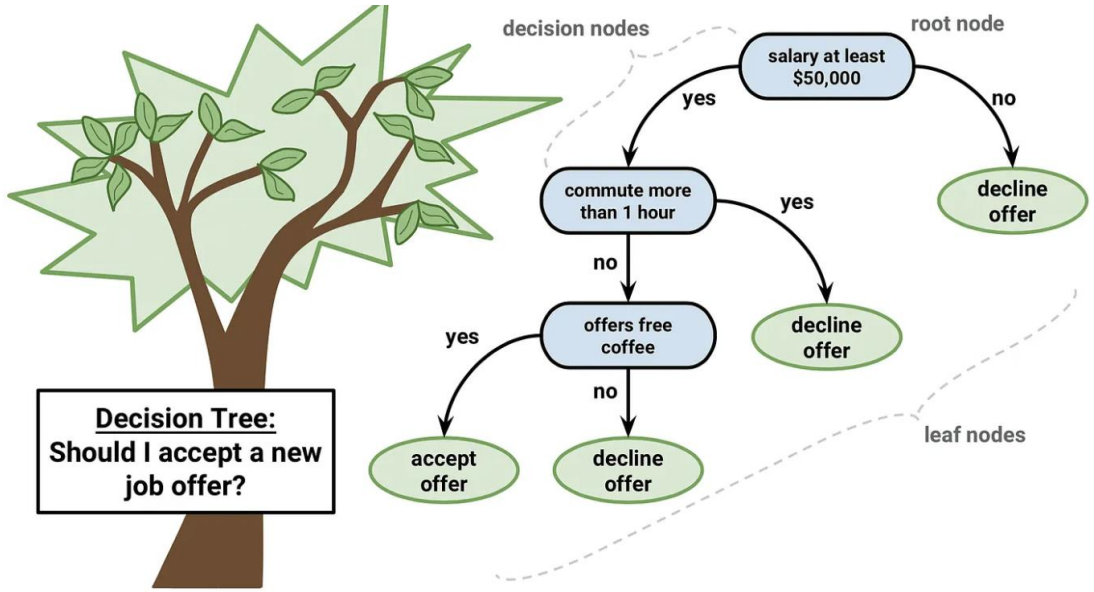
1. Supervised
2. Classification/Regression
3. Minimize Gini Impurity
4. Root Node is Variable with Lowest Impurity

Decision Tree

$$Information\ Gain = Entropy(Parent) - \sum \frac{|Child|}{|Parent|} Entropy(Child)$$

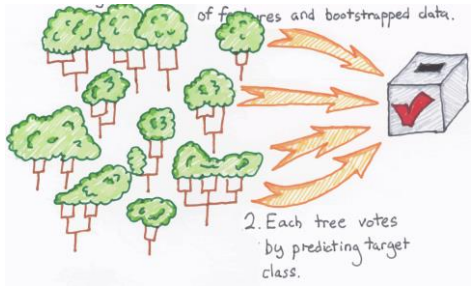
$$Gini = 1 - \sum_{i=1}^{n} p_i^2$$

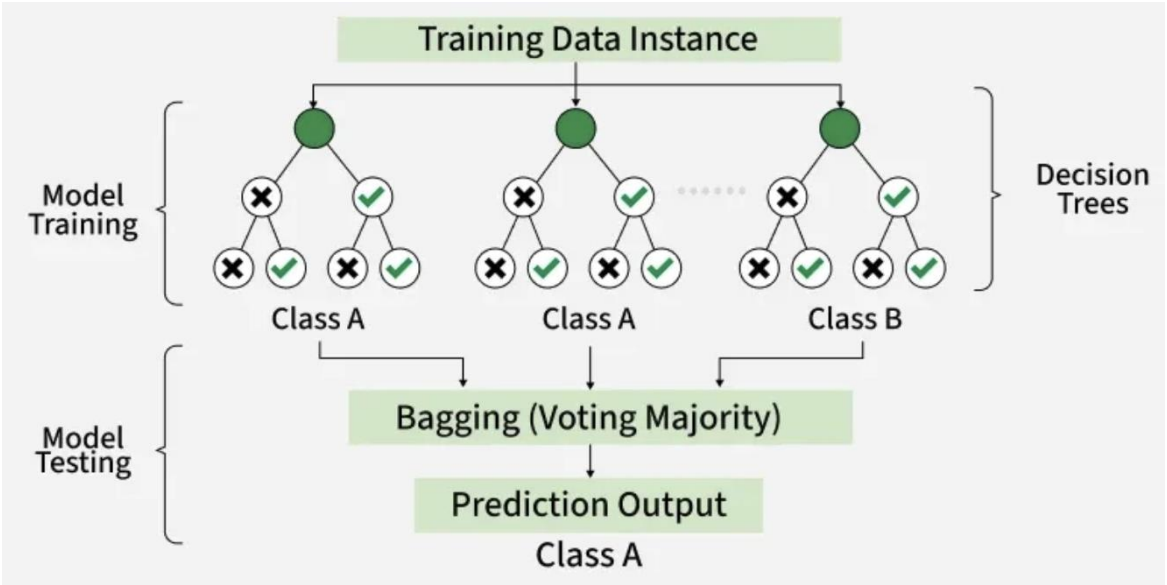
$$Entropy(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$



1. Supervised
2. Classification/Regression
3. Voting Decision Trees
4. More Accurate/Flexible
5. Use Bootstrapped Dataset

Random Forest

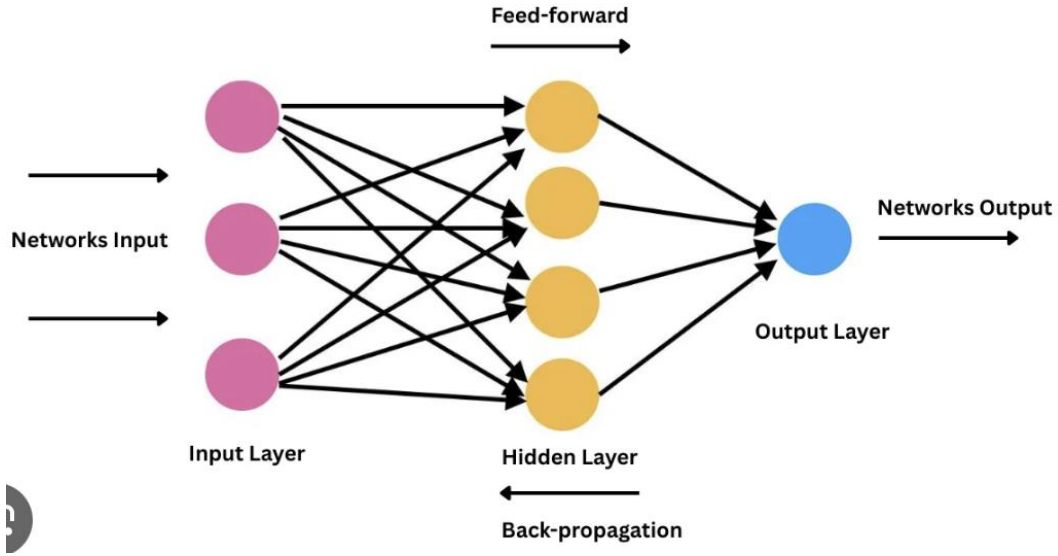


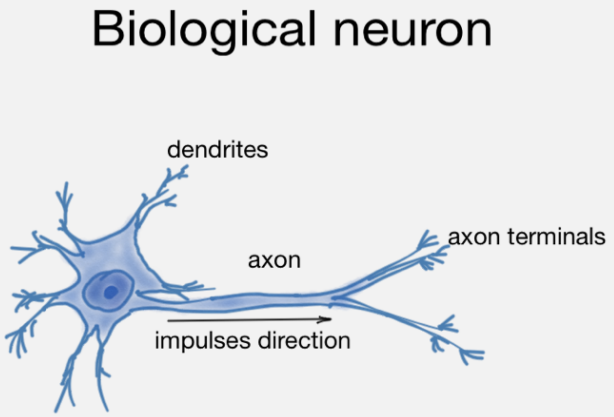


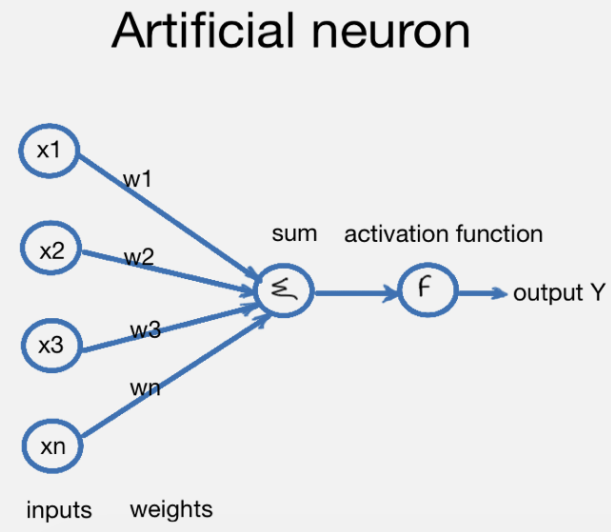
Artificial Neural Networks
Current and Future AI

Artificial Neural Networks

1. Supervised or Unsupervised
2. Brain-Inspired Structurally (McCulloch-Pitts, 1943)
3. Not Brain-Inspired in Learning
4. Discriminative or Generative
5. Perceptron, CNN, RNN, LSTM, GRU
6. Encoder, Transformer, NLP, LLM, Agentic AI
7. Dominate Current AI/Deep Learning

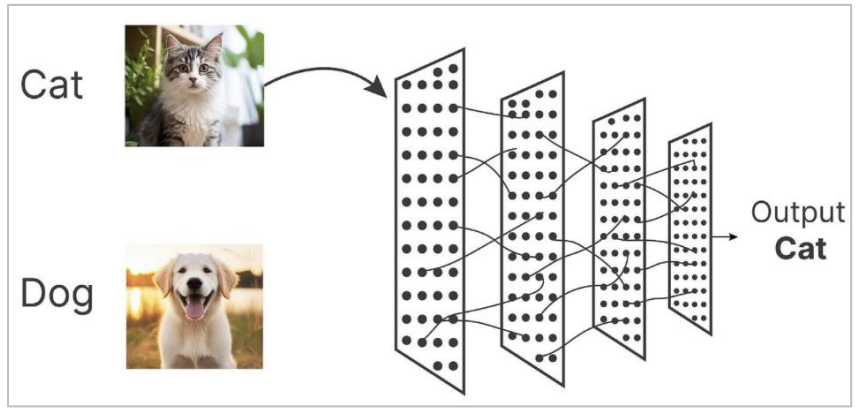




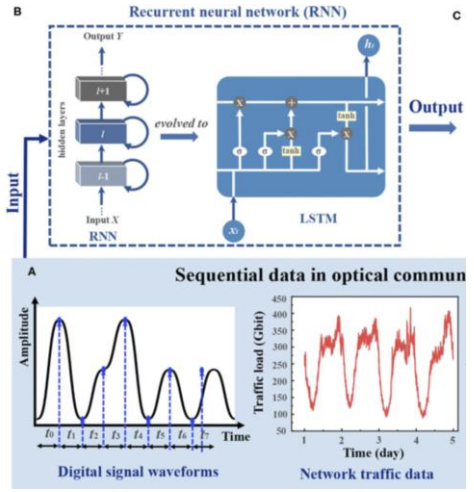


Artificial Neural Networks

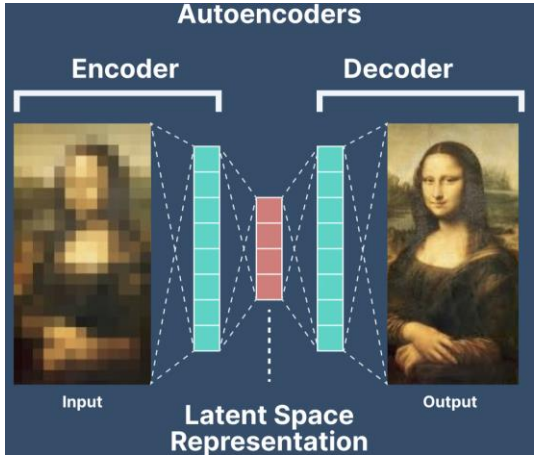
Predicting Neutron Star Crust (Rad Utama et al, 2016)

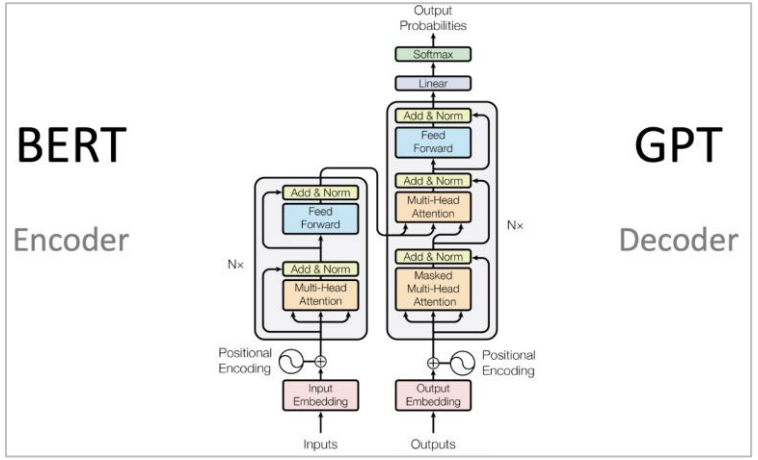


Convolution Neural Networks (CNN) For Image Recognition



Recurrent Neural Networks (RNN) For Sequential Data

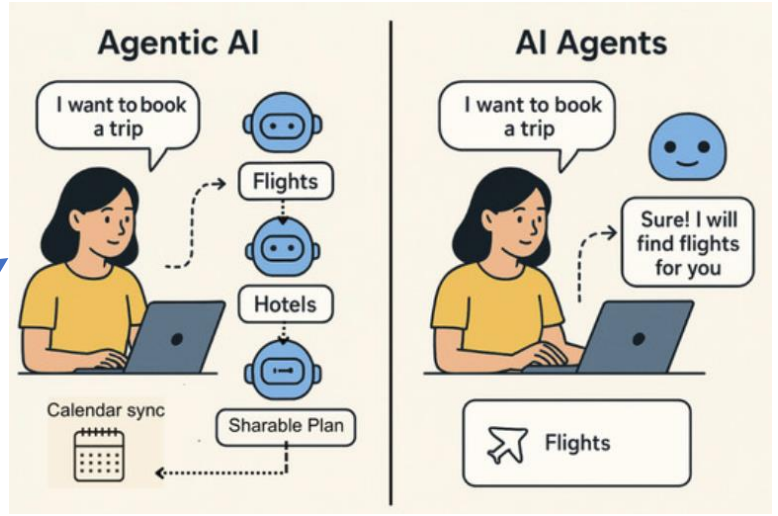




NLP and LLM Empowered by Transformer Self-Attention

Future Direction

Pros



Cons (Minimized using RAG)

Future Direction

Stay Tuned !

Hands-On

Scikit-Learn on Python/JupyterHub (HPC)

Open Your Browser and Go To:

https://bamdev4.cshl.edu/jupyter